The Foundation

0%

Module 1: QUICK START (...

1.1 What You Need (5 Min Set...

1.2 Your First Folder

1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...

2.1 Video Text Guide (Series ...

2.2 One Line of Python Trigge...

2.3 How a 1953 Word Game E...

2.4 The Ladder That Explains ...

2.5 Clawdbot (Moltbot) Has 1...

2.6 Video as Code: My AI Ani...

2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...

2

3.2 Customizing for Your Use ...

3.3 Common Mistakes and H...

2.7 From Nazi Psychology to AI Auditing

13:42

Full paper: [2510.11742] The Ethics Engine: A Modular Pipeline for Accessible Psychometric Assessment of Large Language Models

What You'll Get From This

You will see a real AI system taken apart piece by piece. What you find inside is not magic. It is APIs, async programming, parsing, and statistics. The AI is 10% of the system. The other 90% is engineering that has existed for decades. This is the final lesson in the series (for now) and it ties everything together.

The written version below covers the same ground as the video

Claude or come back to for reference.

The Lesson

A test designed to study authoritarians, now running on AI

I took psychometric tests originally used to measure authoritarian tendencies in the 1950s and aimed them at AI models. But I did more than that. I built a data pipeline that can administer a wide range of personality tests across multiple AI models simultaneously. It can tell you where a model lands on moral foundations, authoritarianism scales, personality dimensions. It can do this across personas, model variations, and different providers.

From the outside, it looks like one intelligence studying another. Something that would require a team of researchers or grants. But I am not bringing this up to show you the results. I want to take it apart. Because something becomes visible when you deconstruct systems that stays hidden when you only watch them get assembled.

What "agent" actually means

Before we open the system, we need to deal with a word that has taken on a lot of weight: agent.

We hear about AI agents that browse the web, write code, manage calendars, analyze documents. The word implies autonomy. Decision-making. Something that can act on its own behalf.

But language is running ahead of the engineering. What we call an AI agent in the vast majority of cases is not a single entity making autonomous decisions. It is a collection of prompts, structured and sequenced by traditional code, that causes a model to behave differently at different stages of a workflow.

The model itself is not deciding to take the next step. The

on conditions that a human engineer defined.

This maps onto something we already understand. Think about the difference between a function call and a running process. A function takes an input and returns an output. It does not decide what to do next. The program that called the function makes

that decision. Each call to a language model works the same way. Prompt in, text out. The agentic behavior, the appearance of autonomy, emerges from the orchestration code that wraps those calls.

There are systems beginning to emerge where models genuinely operate in autonomous loops. That is a real and meaningful evolution. But it is not what the vast majority of deployed systems are doing today. Conflating the two makes it harder to understand what is actually happening when these systems work.

The orchestration layer: async programming and concurrency

So let us look at what is actually happening. Open the Ethics Engine and the first thing you find is not a model. It is a traffic controller.

The Ethics Engine needs to administer dozens of psychometric items across multiple models simultaneously: Claude, GPT, Llama, Gemini. Each provider has different rate limits, different response formats, different failure modes. So there is an orchestration layer that manages all of this. It is written in Python and it is doing something that would be immediately recognizable to anyone who has built a web scraper, a trading system, or a ticket reservation platform.

It manages concurrency. It handles retries when requests fail. It queues prompts and dispatches them in order. It tracks which items have been completed and which are still pending.

None of this is AI. This is async programming. Concurrency management. These are patterns that existed long before language models and they work the same way here as they do anywhere else.

that any of this is happening. It receives a prompt. It returns text. That is the full extent of its involvement in this layer.

How systems talk to each other: APIs

How does the prompt even get to the model? When we say "called an API," there is a concept buried in that sentence that tends to get skipped when people teach this.

An API (application programming interface) is a contract. Two pieces of software agree in advance on the shape of their conversation. What the request will look like. What the response will look like. What is allowed and what is not. Neither side needs to understand the other's internals. They just need to honor the agreement.

When the Ethics Engine sends a prompt to Claude, what actually happens is an HTTP request carrying JSON (a structured text format) that goes to an endpoint at Anthropic's servers. That JSON contains the prompt, the model name, and some parameters. The server processes it and returns JSON containing the model's response.

A structured text message goes out. A structured text message comes back. Everything the Ethics Engine does with that response (the scoring, the analysis, the reliability calculations) happens locally in traditional code.

This is how software has communicated for decades. Web browsers talk to servers this way. Payment systems talk to banks this way. Your weather app talks to a data provider this way. The pattern is not new.

MCP and the pattern: USB, HTTP, LSP, and now AI tooling

Here is where a familiar problem resurfaces. What happens when your system needs to talk to not just one service but twenty? A database, a file system, a web browser, a code interpreter, a calendar, a search engine. If every connection is custom-built, you have a scaling problem. Every new tool

unworkable fast.

We have seen this problem before. Every time, we found the same solution: standardize the interface.

Before USB, every peripheral had its own connector. Printers, keyboards, mice, cameras. All different cables, all different protocols. The solution was one connector, one protocol. Any device that speaks it can connect to any computer that speaks it.

Before HTTP, applications each had their own way of communicating. The solution was one set of rules for requesting and delivering documents. The web became possible.

Before the Language Server Protocol, every code editor needed a custom integration for every programming language. Syntax highlighting, error checking, autocomplete, all built separately for each editor and each language. The solution was to standardize the interface.

This is where MCP (Model Context Protocol) comes in. Introduced by Anthropic in late 2024, explicitly inspired by the Language Server Protocol. The people who built it recognized the pattern: we already solved this problem for programming languages talking to editors, so let us solve it the same way for AI talking to tools.

The architecture has three parts. A host (the AI application), a client (handles the connection), and a server (wraps the external tool and speaks MCP). The model does not call APIs directly. The orchestration layer determines what tools to use. The MCP client manages the connection. The MCP server translates between the protocol and whatever the tool needs internally.

The data and the math: parsing, scoring, statistics

Back to the Ethics Engine. The model returned text. Here is what happens next, and this is where the ratio becomes clear.

The model's response is a string. Natural language.

What bridges it is parsing. Pattern matching. Regular expressions. Text processing tools that have existed for decades.

The number enters the scoring logic. Psychometric instruments have specific scoring keys developed and validated over years of clinical research. Some items are reverse-scored (a response of 5 gets recorded as a 1, because the question was phrased in the negative direction). Subscales are calculated by aggregating specific items. This is validated, peer-reviewed arithmetic.

From there, reliability testing. Cronbach's alpha for internal consistency. Test-retest correlation for temporal stability. Statistical methods implemented in libraries like SciPy and NumPy that predate large language models by decades. The math has not changed since these instruments were validated on human populations.

The real ratio

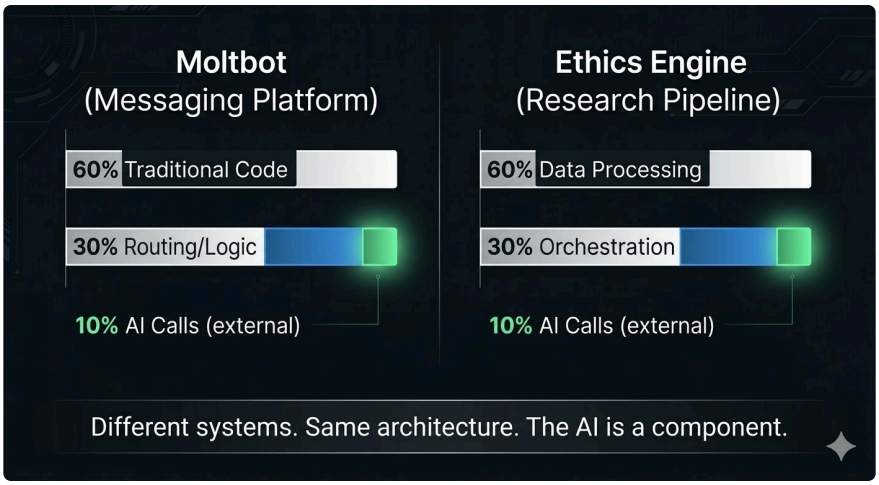
Here is what the Ethics Engine actually looks like when you lay it out:

~10% AI calls — sending prompts, receiving text. This is the only part that involves a language model.

~30% orchestration — async coordination, rate limiting, retry logic, queue management.

~60% data processing — sending and collecting prompts, managing personas, parsing, scoring, analysis, reliability calculations.

This ratio is not unique to the Ethics Engine. It shows up in every well-built AI system. The bad ones ignore it. If you were in **Section 2.5**, you saw the same pattern: Moltbot is roughly 60/30/10 with the AI pushed entirely external. This is the 60/30/10 framework applied to research software instead of a messaging platform. Same ratio. Same architecture. Different domain.



Everything connects

If you have been following this series, the connections are surfacing now.

In **Section 2.2**, we traced a line of Python down through seven layers of abstraction to electrons that are fundamentally probabilistic. In **Section 2.3**, we traced memory from registers and cache through to context windows and system prompts, and found that the template pattern (blanks filled according to rules) holds at every level. In **Section 2.4**, we saw what happens when people build at the wrong layer. In **Section 2.5**, we opened a 100K-star project and found zero AI inside. In **Section 2.6**, we watched one person replace a production team using specs, components, and a pipeline.

Now we have taken an AI "agent" apart from the top and found that most of its layers are traditional engineering. APIs that follow the same request-response pattern software has used for decades. Protocols that standardize interfaces the same way USB and HTTP did before them. Data processing that runs on statistical libraries older than the models they evaluate.

There is a narrative that says AI changes everything. There is a

AI extends what we have always been doing. It adds a layer where natural language becomes the interface and context becomes the program. But that layer sits on top of the same engineering tradition that has been building since we first figured out how to route electricity through logic gates.

The new thing is real. The semantic layer, the ability to operate on meaning rather than syntax, is genuinely novel. But it does not replace the layers beneath it. It depends on them.

What comes next

We are approaching a moment where genuine agentic behavior (models operating in autonomous loops, making decisions that were not predetermined by orchestration code) starts to become real. When that happens, the engineer's challenge will not be making the AI smarter. It will be building the infrastructure around it. Reliability patterns. Error handling. Protocols for safe autonomous operation.

The same work, in other words, that has accompanied every new layer of the stack since Grace Hopper had a compiler that no one believed would work.

The Ethics Engine can tell you the psychological profile of an AI model. That is a real and useful capability. It works because each part does what it is good at. The AI processes natural language and generates responses shaped by context. The traditional code handles orchestration, computation, and reliability. The protocols let different systems communicate without needing to understand each other's internals.

That is not less impressive than magic. It is more impressive, because you can actually understand it. And it actually works.

Go Deeper

- This is the final lesson in the Abstraction series. For the overview and links to every lesson, go back to **Section 2.1**
- The 60/30/10 ratio showed up here and in **Section 2.5**. The full **60/30/10 Framework course** is being built currently in

- The Ethics Engine paper is linked above. You can download it and give it to Claude to explain any section. That is exactly how the docs in **The Vault** are designed to be used.

- The orchestration concepts in this lesson (async, concurrency, rate limiting) are the same patterns running underneath the folder architecture from **Section 1.2**. The folder structure is orchestration in plain English. This lesson shows the same orchestration in Python.

Next Up: Section 3 — Folder Architecture (The Full Method)

The Abstraction series gave you the ideas. The stack. The memory model. The architecture patterns. The ratio. Section 3 takes you back to the practical: the full folder architecture method, how to customize it, and the mistakes people make along the way.

Matthew Creamer

7 24d • Lesson Check-Ins

Foundations 2.7 Check-In

This one traces AI auditing instruments back to places most people don't expect. Vote below, then tell us in the

Poll 103 members have voted

38 30

New comment 3h ago